1260. Shift 2D Grid

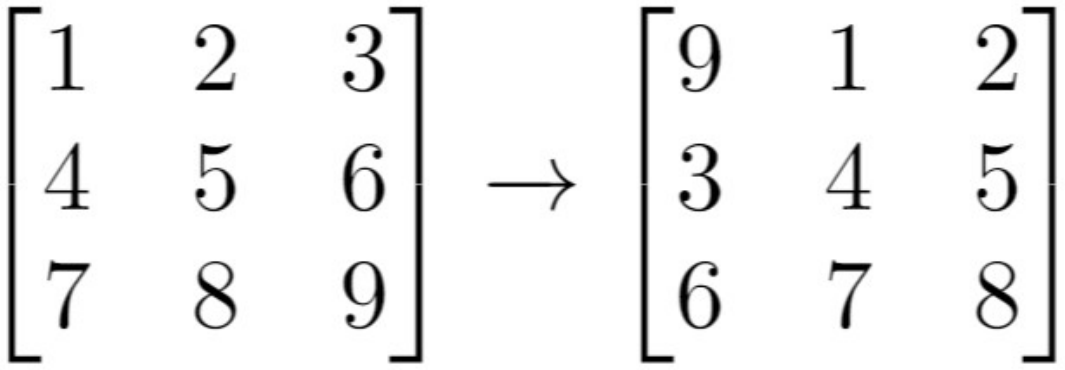
Given a 2D grid of size m x n and an integer k. You need to shift the grid k times.

In one shift operation:

- Element at grid[i][j] moves to grid[i][j + 1].
- Element at grid[i][n - 1] moves to grid[i + 1][0].
- Element at grid[m - 1][n - 1] moves to grid[0][0].

Return the *2D grid* after applying shift operation k times.

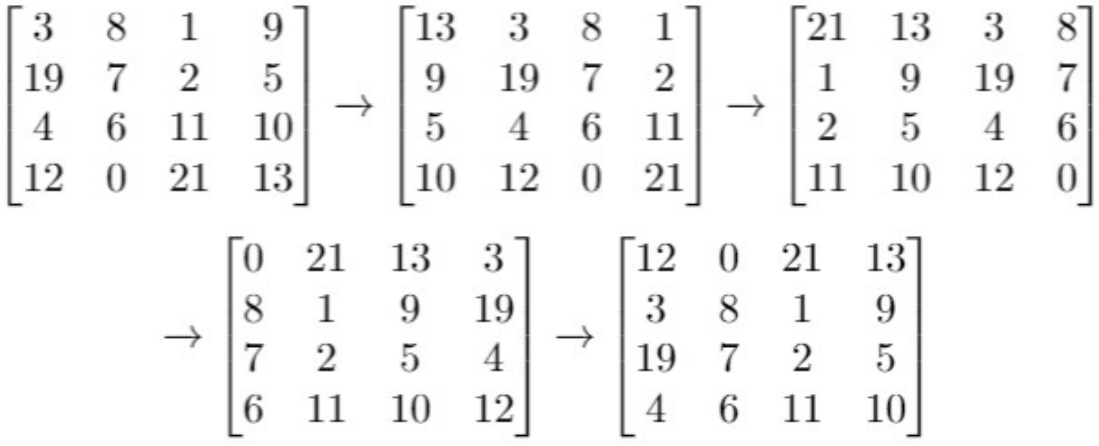
Example 1:



Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 1

Output: [[9,1,2],[3,4,5],[6,7,8]]

Example 2:



Input: grid = [[3,8,1,9],[19,7,2,5],[4,6,11,10],[12,0,21,13]], k = 4

Output: [[12,0,21,13],[3,8,1,9],[19,7,2,5],[4,6,11,10]]

Example 3:

Input: grid = [[1,2,3],[4,5,6],[7,8,9]], k = 9

Output: [[1,2,3],[4,5,6],[7,8,9]]

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m <= 50
- 1 <= n <= 50
- -1000 <= grid[i][j] <= 1000
- 0 <= k <= 100

In-place Rotation | 2D-1D Index Mapping | Beats 100%

```
class Solution {
public:
  vector<vector<int>> shiftGrid(vector<vector<int>>& grid, int k) {
    if (!k) return grid;
    int r = grid.size(), c = grid[0].size();
    int n = r * c;

    k = k % n;
    if (!k) return grid;

    auto shift = [&](int i, int j) {
      while (i < j) {
```

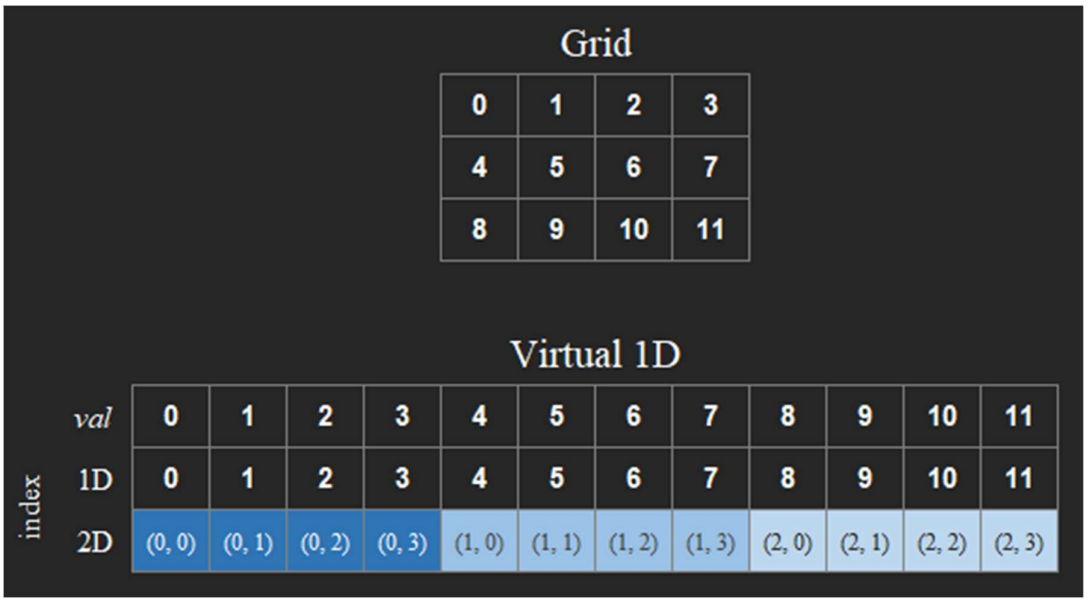
```
            swap(grid[i / c][i % c], grid[j / c][j % c]);
            i++;
            j--;
          }
        };

        shift(0, n - 1);
        shift(0, k - 1);
        shift(k, n - 1);

        return grid;
    }
};
```

Example:

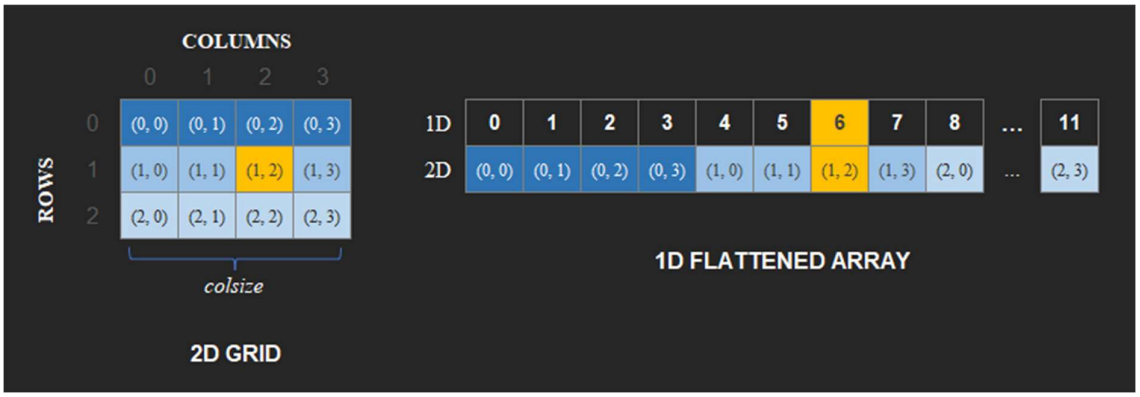
- Shift the grid by 3:



The values are equal to the 1D index for simplification.

Index Mapping

Let's recall how the 2D grid coordinates virtually maps into a 1D flattened array.

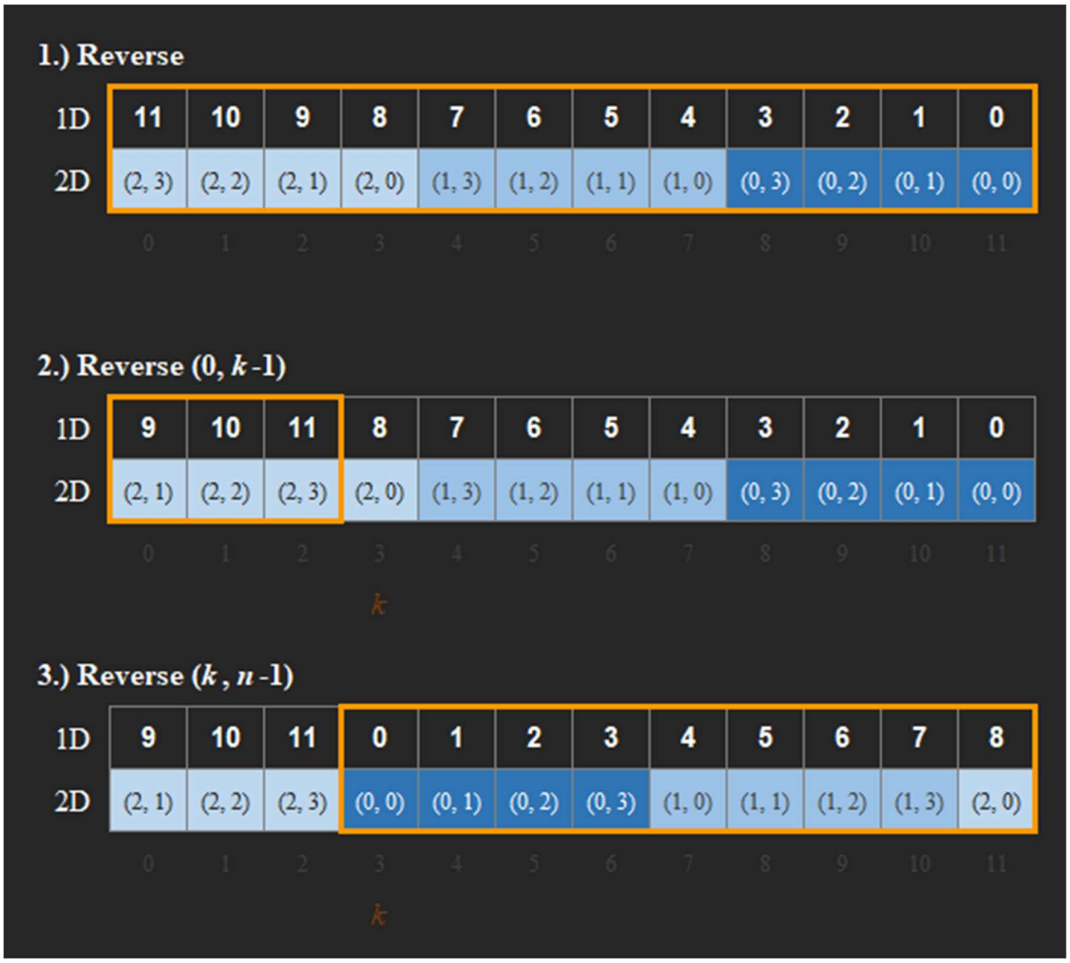


To find the linear index from grid coordinate (row, col), and colsize:

index(1D) = (row * colsize) + col

In-place Shifting

Requires 189. Rotate Array Intuition.



The ***shift(I, j)*** function takes in ***linear index i*** and ***j***, then we swap their values.

The indexes are remapped to 2D grid.

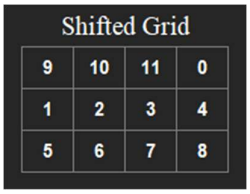
Remapping 1D to 2D

To find the (row, col) from a linear index:

row = [index(1D) / colsize]

col = index(1D) % colsize

Return the ***shifted grid***:



Time Complexity: O(n) **Space Complexity: O(1)**

JAVA:

```
class Solution {
  private void reverse(int start, int end, int[] arr) {
    while (start < end) {
      int t = arr[start];
      arr[start] = arr[end];
      arr[end] = t;
      start++; end--;
    }
  }
  public List<List<Integer>> shiftGrid(int[][] grid, int k) {
    int r = grid.length;
    int c = grid[0].length;
    int n = r * c;
    int index = 0;
    k = k % n;

    List<List<Integer>> list = new ArrayList<>();
    int[] arr = new int[n];

    for (int i = 0; i < r; ++i) {
      for (int j = 0; j < c; ++j) {
        arr[index++] = grid[i][j];
      }
    }
    reverse(0, n - 1, arr);
    reverse(0, k - 1, arr);
```

```
        reverse(k, n - 1, arr);


        index = 0;
        for (int i = 0; i < r; ++i) {
          List<Integer> row = new ArrayList<>();
          for (int j = 0; j < c; ++j) {
            row.add(arr[index++]);
          }
          list.add(row);
        }


        return list;
    }
}
```